

# Nexus Machine: An Energy-Efficient Active Message Inspired Reconfigurable Architecture

Rohan Juneja  
rohan@comp.nus.edu.sg  
School of Computing  
National University of Singapore

Pranav Dangi  
dangi@comp.nus.edu.sg  
School of Computing  
National University of Singapore

Thilini Kaushalya Bandara  
thilini@comp.nus.edu.sg  
School of Computing  
National University of Singapore

Tulika Mitra  
tulika@comp.nus.edu.sg  
School of Computing  
National University of Singapore

Li-Shiuan Peh  
peh@comp.nus.edu.sg  
School of Computing  
National University of Singapore

## Abstract

Modern reconfigurable architectures are increasingly favored for resource-constrained edge devices as they balance high performance, energy efficiency, and programmability well. However, their proficiency in handling regular compute patterns constrains their effectiveness in executing irregular workloads, such as sparse linear algebra and graph analytics with unpredictable access patterns and control flow. To address this limitation, we introduce the *Nexus Machine*, a novel reconfigurable architecture consisting of a PE array designed to efficiently handle irregularity by distributing sparse tensors across the fabric and employing active messages that morph instructions based on dynamic control flow. As the inherent irregularity in workloads can lead to high load imbalance among different Processing Elements (PEs), *Nexus Machine* deploys and executes instructions en-route on idle PEs at run-time. Thus, unlike traditional reconfigurable architectures with only static instructions within each PE, *Nexus Machine* brings dynamic control to the idle compute units, mitigating load imbalance and enhancing overall performance. Our experiments demonstrate that *Nexus Machine* achieves 90% better performance compared to state-of-the-art (SOTA) reconfigurable architectures, within the same power budget and area. *Nexus Machine* also achieves 70% higher fabric utilization, in contrast to SOTA architectures.

## Keywords

Reconfigurable Computing, Dataflow Architectures, Sparse Workloads, Graphs, Active Messages

## ACM Reference Format:

Rohan Juneja, Pranav Dangi, Thilini Kaushalya Bandara, Tulika Mitra, and Li-Shiuan Peh. 2025. Nexus Machine: An Energy-Efficient Active Message Inspired Reconfigurable Architecture. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3725843.3756091>



This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/2025/10

<https://doi.org/10.1145/3725843.3756091>

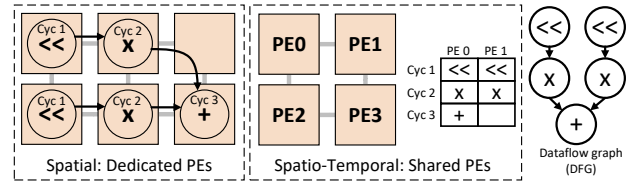


Figure 1: Spatial vs Spatio-temporal

## 1 Introduction

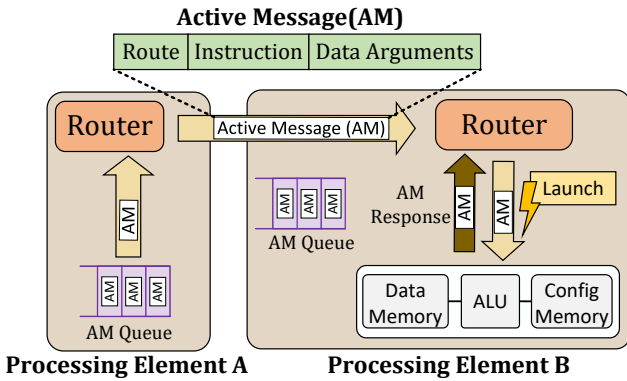
Edge devices require high performance within constrained power budgets. ASIC accelerators are efficient for specific applications, but reconfigurable computing offers a flexible alternative with lower engineering costs. A reconfigurable architecture consists of many processing elements (PEs) connected by an on-chip network. Field Programmable Gate Arrays (FPGAs) are the most successful reconfigurable architectures today; however they suffer from sub-optimal compute density and energy efficiency due to bit-level reconfigurability. In contrast, modern reconfigurable architectures provide word-level reconfigurability, balancing adaptability and efficiency with significant research targeting both edge [15, 16, 33, 35, 36] and server domains [5, 40, 46, 48].

A Coarse-Grained Reconfigurable Architecture (CGRA) uses simple PEs connected by a reconfigurable on-chip network to form high-throughput datapaths. Each PE contains an ALU, router, and reconfiguration memory. Data is loaded from a global scratchpad memory (SPM) to the PE array for execution, and results are stored back in the SPM. As shown in Fig. 1, CGRAs are classified into Spatial and Spatio-temporal [25]. Spatial architectures [15–17, 41] maintain a fixed mapping of compute and communication, ensuring low configuration energy but potentially lower performance. Spatio-temporal [33, 36, 49, 52], on the other hand, allow each PE to reconfigure to a new instruction every cycle, enhancing performance but potentially increasing energy consumption [2]. These architectures excel in accelerating regular workloads with predictable access patterns and minimal control flow, such as DSP workloads including dense linear algebra.

Critical workload domains for edge devices, such as edge AI, AR/VR, electric vehicles (EVs), gaming, and IoT, exhibit irregular computational kernels like sparse linear algebra and graph analytics. These workloads become especially challenging with dynamic

sparsity [63], such as activation and weight pruning in neural networks [31], where both matrices are defined at runtime. Current reconfigurable architectures struggle with irregular workloads, characterized by unpredictable memory accesses and complex control flow. These challenges arise because such dataflows are not well-suited for static compile-time instruction and data mapping techniques typically employed by these architectures. Irregular memory accesses lead to significant bank conflicts, limiting memory-level parallelism, while irregular control flow causes serious load imbalance, leaving most PEs idle at runtime.

Recent works aim to tackle irregular workloads, but achieving high performance with energy efficiency remains challenging. Modern reconfigurable architectures like Triggered Instruction Architecture (TIA) [45] leverage a data-local execution model for edge devices. Unlike CGRAs with centralized data memory, each PE in TIA has its own distributed memory and configuration memory. TIA operates by dispatching operands to a PE, triggering the loading of a statically placed instruction. These architectures rely on a runtime scheduler for tag matching and a priority encoder to trigger the next instruction, adding significant hardware overhead. Due to the irregular nature of workloads, these architectures face difficulties with load balancing and efficient fabric utilization.



**Figure 2: Active Message (AM) communication mechanism:** An AM originating from PE A is launched at destination PE B, where it is executed on the ALU, and interacts with the data and configuration memory. Additional AMs may be generated in response, if necessary.

To address these challenges, we introduce *Nexus Machine*, a novel reconfigurable architecture inspired by the active message paradigm. **Active Message (AM)** [12] is a communication mechanism that enables flexible and efficient task execution by leveraging the proximity of tasks to relevant data. As shown in Fig. 2, each AM contains routing information, instructions for the target node, and data payload for arguments. An AM originating from PE A traverses the on-chip network to reach PE B, where it launches computation on the ALU, interacting with data and configuration memory. PE B can respond by generating additional AMs.

*Nexus Machine* efficiently executes irregular workloads by leveraging the active message paradigm, addressing irregular memory accesses and load balancing. By distributing sparse tensors across

the fabric and performing data-local execution, *Nexus Machine* avoids expensive bank conflicts and enhances memory access. Instead of executing computations immediately on the source PE, the architecture postpones execution along the datapath, allowing idle PEs to process computations as messages traverse the network. It provides a specialized compiler framework and hardware for *Data-driven execution* of sparse tensors, optimizing memory accesses. To improve load distribution and system efficiency, *Nexus Machine* introduces a novel load-balancing strategy for In-Network execution of AMs, dynamically deploying and executing AMs en-route.

Compared to contemporary platforms, while CPUs and GPUs are often favored for their flexibility and ease of programming, they are less efficient for highly irregular workloads compared to domain-specific architectures. GPUs perform well with minimal control divergence, but optimizing them for irregular workloads remains an active research area [3, 42]. Similarly, High-Level Synthesis (HLS) for FPGAs is optimized for regular patterns, limiting its effectiveness for irregular workloads, a key focus of ongoing research [13, 38]. Unlike FPGAs, which reconfigure at the gate level with a bitstream, *Nexus Machine* reconfigures its ALUs instruction-by-instruction each cycle, offering faster reconfiguration times. Additionally, unlike some previous CGRAs [60] that use LUTs with ALUs within their PEs for finer control, *Nexus Machine* does not use LUTs. Our contributions include:

- An architecture and compiler framework for *Nexus Machine*, showcasing its applicability in sparse tensor computations.
- An exhaustive exploration of *Nexus Machine*'s architectural parameters through a cycle-accurate simulator, covering diverse workloads.
- Implementation of *Nexus Machine* in 22nm process with compiled memories, outperforming a *generic CGRA* baseline by 1.9x and achieving 1.7x better fabric utilization.

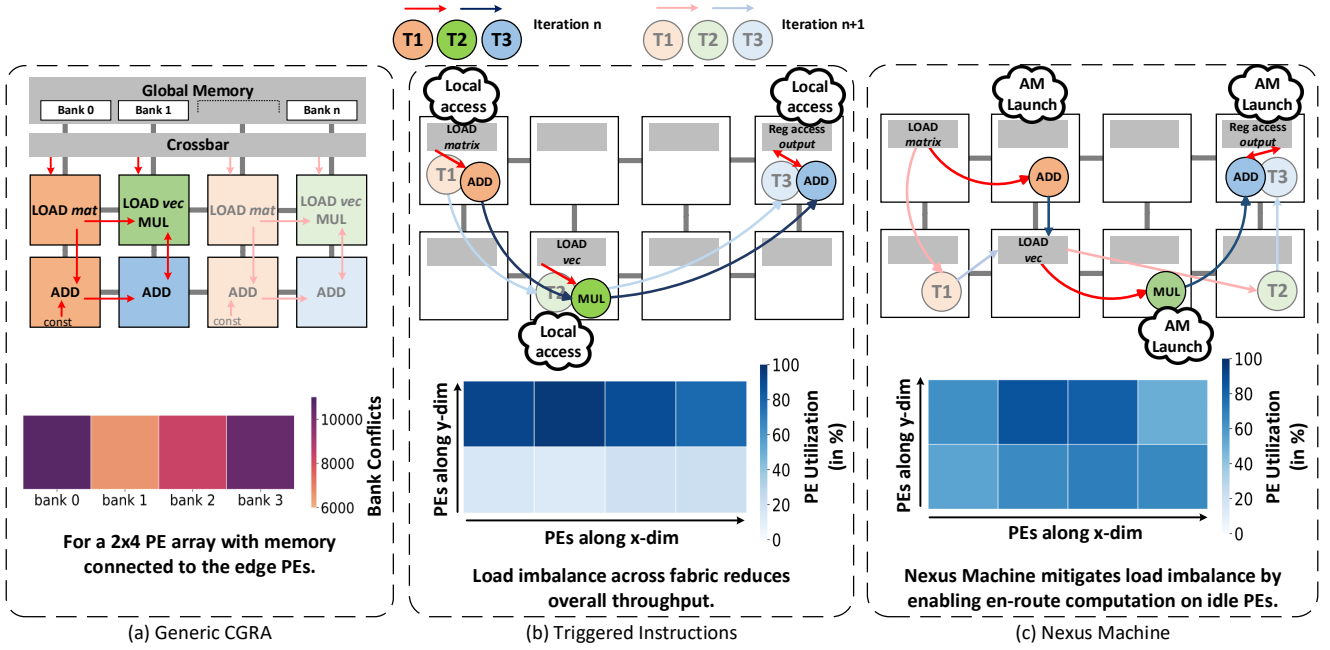
## 2 Background and Motivation

### 2.1 Active Messages and Task Migration

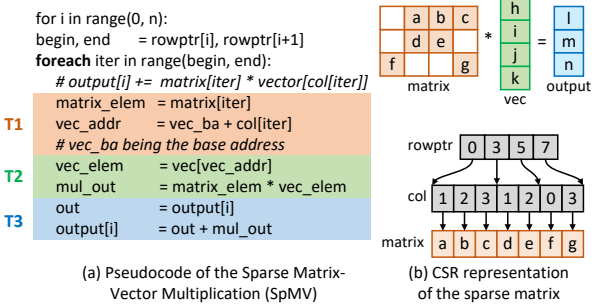
Active Messages (AM) [12] was proposed as a lightweight communication scheme that sends network messages from one machine to another, with messages triggering execution of user-defined functions upon arrival. These lightweight functions can access memory, invoke computations, and dispatch further active messages as a response. The core concept behind active message revolves around dispatching computation to the data location rather than transferring data to the computation. This effectively removes coherency overhead and minimizes data movement.

AMs have been realized and implemented in a range of multi-computers in the past. In [12], a practical implementation of AM on CM5 and n-CUBE/2 architectures was presented. The J-Machine [10] is a notable parallel processor featuring comprehensive hardware and instruction support for active messages. It facilitates the remote spawning of fine-grained tasks at a granularity of approximately 20 instructions. Also, commercially available processors like the Cray T3D and its subsequent iterations have leveraged network designs similar to that of the J-Machine.

With *Nexus Machine*, we propose a low-power reconfigurable architecture that is inspired by the active message model to target irregular workloads. Our definition of AM diverges significantly



**Figure 3: Program execution comparison for SpMV kernel, illustrating two consecutive iterations with a bank conflict.** (a) *Generic CGRA*: Data flows through statically placed instructions (top) and bank conflicts across various banks for a real workload with  $n=2048$  on a  $2 \times 4$  PE array (bottom) (b) *Triggered Instructions*: Illustrates data-local execution with messages, invoking tasks at the location of data, reducing data movement (top) and visual representation of the load imbalance across the PE array (bottom) (c) *Nexus Machine*: Enhances performance and PE utilization through a unique approach - enabling opportunistic execution and utilizing idle ALUs for en-route instruction execution (top) and visual representation of the uniform load balance across the PE array (bottom). Data movements are represented by red arrows, while blue arrows depict message transfers.



**Figure 4: Sparse Matrix-Vector Multiplication (SpMV).**

from the original concept in its application to reconfigurable architectures. First, we store the instruction itself within the message, rather than relying on a handler to trigger a set of instructions, to ensure immediate execution of the instruction upon arrival at a PE. *Nexus machine* also leverages the static compilation strategy of previous reconfigurable architectures, with the software compiler first intelligently placing instructions and data across the fabric. Unlike previous architectures, where the data flows through statically placed instructions on PEs, *Nexus Machine* reduces data movement by sending and executing instructions at the location where the data resides.

In the *Nexus Machine* architecture, the entire execution is initiated and orchestrated by the data; thus we refer to it as **Data-Driven Execution**. Unlike prior multi-computer systems with high bandwidth networks between machines, *Nexus Machine* employs lightweight Networks on Chip (NoCs) between simple ALUs, similar to previous reconfigurable architectures, which facilitates frequent message passing to initiate ALU operations, making the *Nexus Machine* well-suited for the *Data-Driven Execution* model.

## 2.2 Challenges of Irregular Workloads on reconfigurable architectures

Current reconfigurable architectures excel in creating static software pipelining, where a task is divided into instruction-level pipeline stages, and gets mapped to the PEs across the fabric. To execute tasks, data moves from one PE (producer) to the next PEs (consumers, data-dependent on the producer) along fixed (compile-time) pre-scheduled datapaths. This works well for regular workloads where the data access and control flow are predictable.

However, mapping irregular workloads on these architectures significantly impacts performance, fabric utilization, and energy consumption for data movement. To illustrate this issue with an example, we use **Sparse Matrix Vector Multiplication (SpMV)** kernel that multiplies a sparse matrix *matrix* with a dense vector *vec*. Fig. 4(a) provides a pseudo-code for SpMV at the instruction

level. The sparse matrix in Fig. 4(b) is stored in a commonly used compressed sparse row (CSR) format to encode the row and column indices of the non-zero elements. *rowptr* points to the first non-zero element information for each row in the *col* vector, which in turn stores the column indices of all the non-zero elements for each row consecutively. SpMV is an irregular workload due to its use of multiple levels of indirection to access the data: *rowptr* to access *col*, which in turn helps access a specific index in *vec*.

Fig. 3(a) (top) shows the mapping of SpMV on a *generic CGRA* that has a global memory shared across all the PEs. Here the operations are statically mapped onto the PEs and the routing path of the data between dependent PEs is also fixed (red arrows). Further, to enhance fabric utilization, multiple iterations of the loop are unrolled and spatially mapped. However, this approach exacerbates bank conflicts when dealing with irregular workloads. The architecture's demand for synchronized operation across all PEs in a predictable manner means that any bank conflict results in stalls. For the SpMV workload mapped to an array of 16 PEs connected to 4 memory banks on one edge, Fig. 3(a) (bottom) illustrates the occurrences of conflicts across different memory banks.

To mitigate bank conflicts caused by irregular memory access patterns in global memory, we distribute the memory across PEs, enabling data-local execution that serializes memory accesses at each PE. To convert SpMV into a sequence of tasks suitable for data-local execution, we divide the code at each memory operation. This is illustrated for SpMV by the different colored segments in Fig. 4(a). Task **T1** accesses the *matrix* array (represented by *rowptr* and *col*), **T2** accesses elements in the *vec* array, and **T3** executed on the corresponding elements of the *output* array. During the compilation phase, *Nexus Machine* restructures the CSR arrays of the matrix into AM entries (see Section 3.6). Each entry consolidates the matrix data and the locations of vector and output elements, using *rowptr* and *col* arrays. These entries help guide the messages as they traverse the architecture. In *Nexus Machine*, dynamic routing introduces network congestion analogous to bank conflicts in global memory CGRAs, as message routing depends on congestion levels.

The architecture baseline we use for data-local execution resembles that of Triggered Instructions [45] with multiple PEs, where each PE includes a distributed data memory, ALU, and a configuration memory with a scheduler. The tasks are anchored with the data, triggered when messages arrive with the data. Fig. 3(b) (top) illustrates how our transformed SpMV can be executed on such an architecture. **T1** performs local load for the *matrix* element and calculates the address for the *vec* element. When the message reaches the PE with the *vec* element, **T2** executes a local load of *vec* element and multiplies it with the *matrix* element obtained from the remote PE. Finally, when the message reaches the PE with the *output* element, **T3** performs a local aggregation. While our transformed SpMV workload can be mapped to such an architecture, it still faces challenges in terms of performance and resource utilization due to load imbalance. This is because the instructions are anchored with the data, and cannot move at run time to different PEs. For SpMV, these imbalances are evident by the varying numbers of elements aggregated for each *output*. Fig. 3(b) (bottom) represents the load imbalance observed within PE array.

*Nexus Machine* effectively addresses the low utilization and performance challenges posed by irregular workloads by opportunistically executing active messages on the idle PEs en-route. It does this by exploiting the AM paradigm that allows the transmission of not just data operands, but also **instructions** within a single message. The architecture employs **Opportunistic Execution**, a strategy that postpones execution along the datapath instead of executing immediately on the source PE. Operations are executed on the first idle PE encountered along the route, or alternatively, on the destination PE if no idle PEs are available. In Fig. 3(c) (top), we illustrate how *Nexus Machine* applies this approach to execute the SpMV kernel. Task **T1** initiates the local access of the *matrix* element but forwards the computation into the network, where it is executed on the first available idle PE. This generates an AM response with Task **T2**. Similarly, task **T2** is sent into the network with the corresponding *matrix* and *vec* elements and is computed en-route, resulting in another AM response with Task **T3**. Finally, task **T3** performs a local aggregation of the *output* element. We refer to this as **In-Network Computing** for dataflow architectures. Fig. 3(c) (bottom) visually demonstrates how *Nexus Machine*'s coupling of compiler data placement with runtime distribution of instructions across the fabric enables uniform load balance across the PEs, significantly enhancing utilization and performance.

### 3 Nexus Machine

*Nexus Machine* is a novel reconfigurable architecture specialized for irregular workloads that uses AM paradigm for *Data-Driven* execution model. This section introduces the execution model of *Nexus Machine*, followed by its microarchitecture and compiler. Fig. 8(a) provides an overview of the architecture, showcasing PEs interconnected via a mesh network.

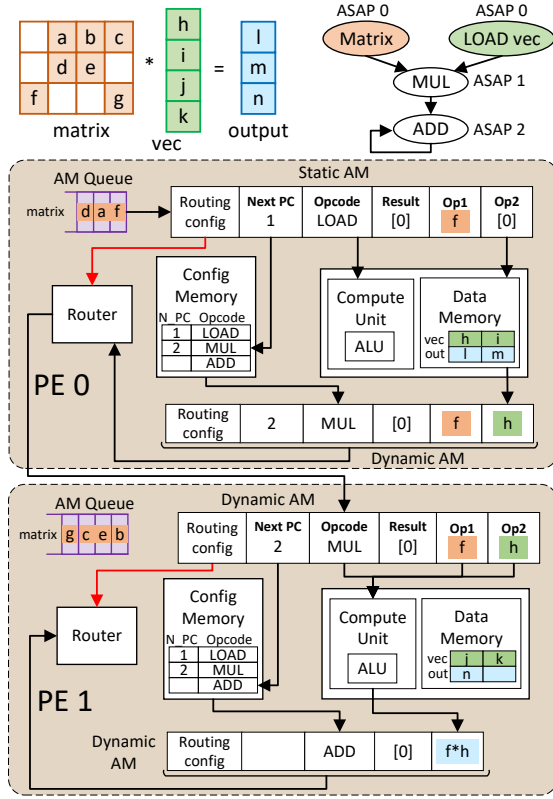
#### 3.1 Execution Model

**Static Initialization.** *Nexus Machine* follows a distributed tensor placement approach, partitioning all tensors across the PEs. Initially, the compiler generates a *static AM* for each element of the first tensor. These *static AMs* are then stored in the active message network interface, while the remaining tensors are placed in the data memory within each PE. Additionally, the compiler generates opcodes corresponding to the workload and stores them in the configuration memories of all the PEs.

**Dynamic Execution.** When execution is initiated, the active message network interface dequeues the first *static AM* and then routes it based on dynamic turn model [37] routing protocol to the PE containing the next operands. Upon arrival at the PE, the AM is decoded and updated accordingly, transforming it into a *dynamic AM* created on-the-fly, in contrast to the predefined *static AMs* generated at compile time. Once the AM has gathered all the required operands, it proceeds towards the next destination PE, for execution. Additionally, we allow these AMs to perform computations en-route if they encounter an available compute unit. The final result gets stored in the memory of the destination PE.

Thus, **static AMs** are generated at compile time, containing the initial instructions for computation. During execution, *static AMs* are transformed into **dynamic AMs**, carrying different instructions based on the AM format and real-time conditions. This allows

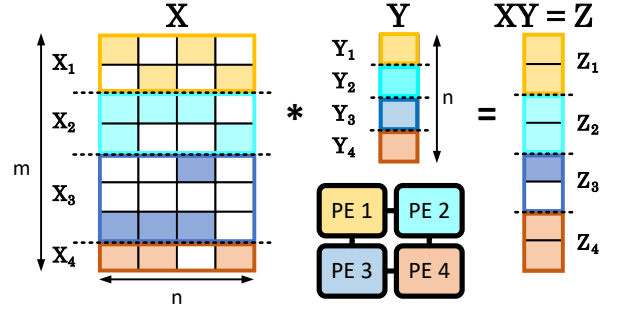




**Figure 5: Execution of SpMV using the example from Fig. 4 on a 2-PE fabric.** The matrix is partitioned and stored as static AMs in AM Queues, while the input vector and output are placed in local data memories. At cycle 0, PE0 processes an AM that loads operand  $h$ , generates a dynamic AM with a MUL opcode, and sends it to PE1. In cycle 1, PE1 performs the multiply, creates an ADD AM, and accumulates the result into output entry  $n$ . Square brackets  $[ ]$  indicate addresses, and red arrows show control signals. PE1's AM queue activity is omitted for clarity.

*Nexus Machine* to optimize both data placement and computation, enhancing efficiency for irregular workloads.

Fig. 5 illustrates the execution of SpMV using the example from Fig. 4 on a fabric with 2 PEs. The *matrix* operand is split and converted into *static* AMs, stored in AM Queues, while *vec* and *output* are divided and placed in *data* memories. At cycle 0, PE0's initial *static* AM (with operand  $f$ ) performs a LOAD to fetch *Op2* ( $h$  in this case) from *data* memory, then creates a *dynamic* AM with *Op2* and *Opcode* MUL. This *dynamic* AM is sent to PE1, where operands are multiplied in cycle 1, and the result is combined with *Opcode* ADD to generate the next AM. PE1 updates *output* ( $n$  in the *data* memory) by adding the multiplied result to it (not shown in the figure). Note that PE1 concurrently dequeues and processes AMs from its AM Queue, although this is not shown here for the sake of simplicity.



**Figure 6: Data partitioning for SpMV involves multiplying a sparse matrix  $X$  ( $m \times n$ ) with a dense vector  $Y$  ( $n \times 1$ ) to produce  $Z$  ( $m \times 1$ ).** Four partitions are shown in different colors, distributed across PEs denoted by the same color. Non-zero elements are shown in colored squares.

**3.1.1 Tensor Partitioning.** *Nexus Machine* employs a coarse-grained distributed tensor partitioning strategy using a dissimilarity-aware approach. Each tensor is segmented across PEs to balance memory utilization and reduce communication overhead. For large tensors exceeding local capacity, tiling [21, 57] decomposes the computation into smaller sub-tensors that fit within on-chip memory.

Fig. 6 illustrates an SpMV example for  $N = 4$  PEs. The sparse input tensor  $X$ , stored in Compressed Sparse Row (CSR) format, is divided into  $X_1, \dots, X_N$ , each with the same column dimension but differing row counts, such that  $\text{nnz}(X_i) \approx \text{nnz}(X)/N$ . This partitioning is computed via a linear scan of the row pointer array, with complexity  $O(m)$ , where  $m$  is the number of rows. The one dimensional tensors  $Y$  and  $Z$  are partitioned correspondingly to maintain alignment with  $X$ . For dense tensors, uniform segmentation into  $k$  equal parts is applied. As described in Sec. 3.6, partitioning plays a critical role in balancing locality and execution efficiency, and is tightly integrated into the compilation flow to reflect sparsity patterns and architectural constraints.

**3.1.2 Data-Driven Execution.** In the *Nexus Machine* architecture, each PE stores the first input tensor as pre-compiled active messages (AM) and a partition of other tensors. At the source PE, the first AM is sent to the PE with the next data element. Since the initiation of execution is triggered by the active message or one of the tensor elements, we refer to this model as *Data-Driven Execution*.

Fig. 8(a) illustrates an example where instruction  $I0$  at PE4 triggers  $I1$ , generating a dynamic AM with  $I1$  and  $I0$ 's output, then sent to destination PE15 for execution. However, restricting execution solely to the source or destination PE results in significant underutilization and workload imbalance across PEs for irregular workloads where data dependencies and control flow are unpredictable.

**3.1.3 In-Network Computing.** To enhance load balancing, we employ *opportunistic execution* approach, allowing an AM to execute on an *intermediate PE* as it travels towards its final destination. With this execution model, en-route AMs carry both the instruction and required data operands, enabling *intermediate PEs* to perform computations whenever an idle ALU is encountered. In Fig. 8(a), the highlighted cores in blue represent the potential *intermediate PEs* for a message routed from PE4 to PE15. The message is executed at

PE13, denoted as *I1'*, as it serves as the first intermediate PE along the route with an idle ALU.

This provides several advantages: (a) It introduces a hardware mechanism to enhance workload distribution and fabric utilization by leveraging idle PEs, (b) reduces NoC contention by deciding whether messages are executed en-route or continue toward their destination PEs, and (c) minimizes the amount of data traversing the NoC by coalescing the updates to the original message and discarding unnecessary data.

**3.1.4 Termination and Global Synchronization.** *Nexus Machine* completes execution when all PEs are inactive and no messages are in transit, generating a global idle signal to indicate completion. *Nexus Machine* uses this global idle signal to notify the host via an interrupt. This approach is suitable for edge architectures with limited memory, where tiling improves resource use and scalability. Data tiles are executed sequentially in a global synchronized manner. Once a tile finishes, the system detects the idle state and triggers the next tile's execution.

### 3.2 Multi-destination active message format

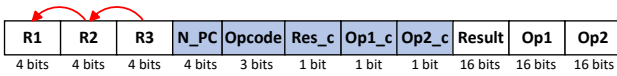


Figure 7: Active Message format

*Nexus Machine* extends the fundamental active message primitives to accommodate a multi-destination based routing mechanism. Fig. 7 illustrates the message format: the first 12 bits specify intermediate destinations (*R1*, *R2*, *R3*), based on our workload analysis (as SDDMM has three inputs, destinations correspond to two inputs and one output tensor). The next 4 bits contain the Program Counter (PC) for the next instruction (*N\_PC*), followed by 3 bits for the *Opcode*. A single bit (*Res\_c*) indicates if the message carries a result. The subsequent 2 bits (*Op1\_c* and *Op2\_c*) identify whether *Op1* and *Op2* are addresses or values. Depending on *Res\_c*, the *Result* field contains the final result or its address, while the next 16 bits hold data for Operand1 (*Op1*) and Operand2 (*Op2*).

When the message reaches its first destination (*R1*), it is routed through the LOCAL port of the corresponding PE and forwarded to its ALU for execution. The ALU processes the associated computation payload, after which the message is reassembled with an updated destination list. The remaining destinations are cyclically rotated, making *R2* the first and *R3* the second. In the *Nexus Machine*, a message is equivalent to a packet or flit (all messages are a single-flit packet).

### 3.3 Nexus Machine Microarchitecture

As presented in Fig. 8(a), the *Nexus Machine*'s fabric comprises processing elements (PEs) interconnected with a mesh network, with a global termination detector. Each PE is linked to four neighboring PEs in North, East, South, and West directions. The off-chip memory is connected to the four PEs located along the left edge.

**3.3.1 Processing Elements (PEs).** As shown in Fig. 8(b), each PE has a compute unit, a dynamic router for network connectivity

with congestion control, a load/store unit, and two Network Interface logic. *Input Network Interface* unit is responsible for efficiently handling incoming AMs from the NoC, while the AM Network Interface unit initiates the injection of new messages into the NoC.

**Input Network Interface.** The *Input Network Interface* unit manages incoming AMs to a PE. Depending on the message: (a) If it pertains to an ALU operation, it is directed to the *Compute Unit* for execution; (b) Alternatively, in case of a memory operation, the message is forwarded to the *Decode* unit. This unit initiates a load or store operation, utilizing the operand address information (*Op1* or *Op2*) contained in the message. Once these operations are completed, the resulting *output dynamic AM* is dispatched to the *AM Network Interface* for injecting into the network.

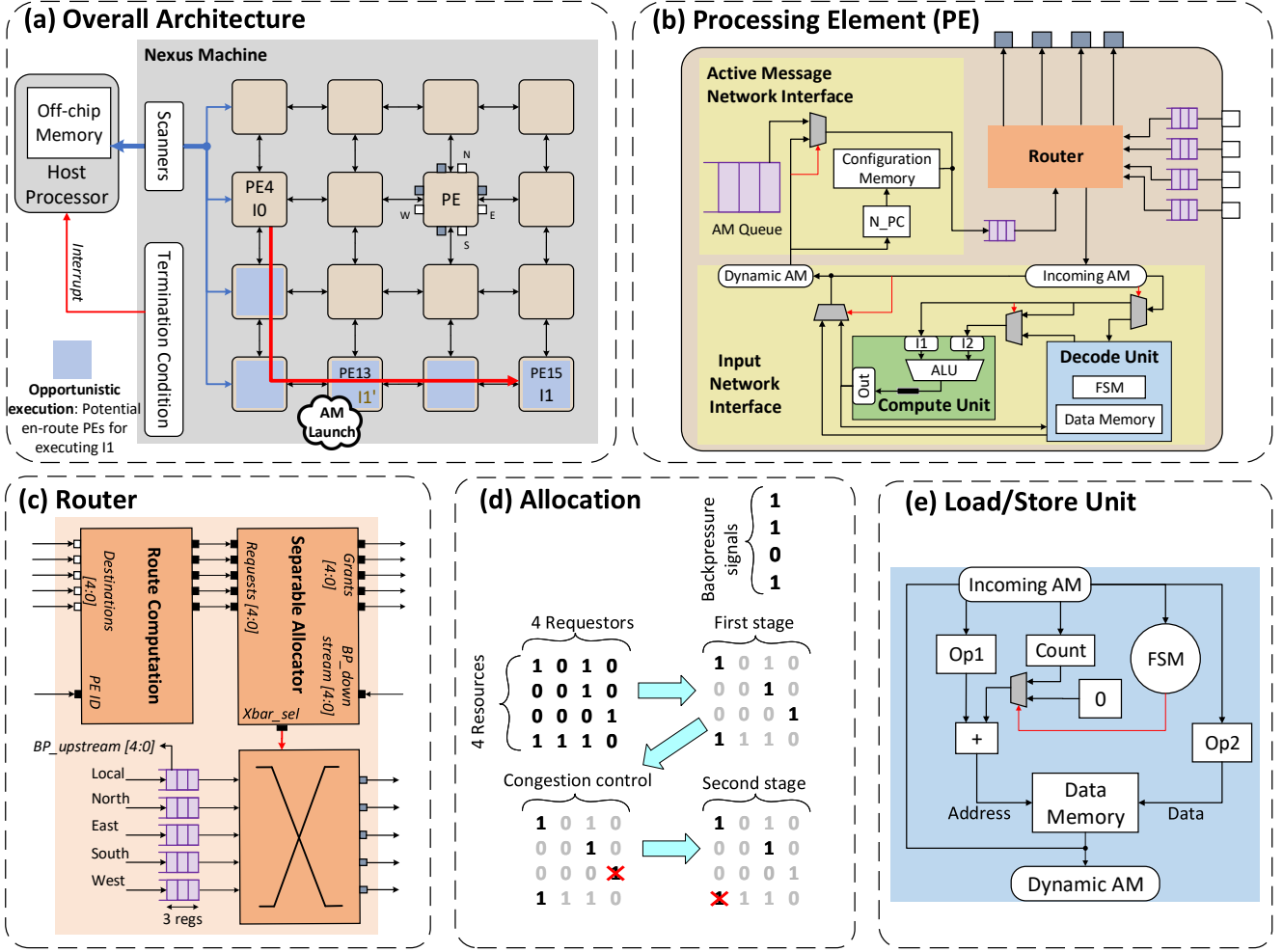
**Compute Unit.** The *compute unit* within a PE can perform 16-bit arithmetic operations, logic operations, multiplication, and division on its ALU. An incoming AM at the *Input Network Interface* dispatches two operands, *Op1* and *Op2* along with the *Opcode* field in the message to the compute unit. After computation, it generates an output that is combined with the original AM, replacing the *Op1* field in the message. Finally, this modified AM is forwarded to the *AM Network Interface* for injecting into the network.

**Load/Store Unit.** The *Load/Store Unit*, as shown in Fig. 8(e), can be flexibly configured to operate in dereference and streaming modes. In **dereference mode**, the operand address field (*Op1* or *Op2*) in the message triggers the loading of a single element. This gets embedded into the *output dynamic AM*. Conversely, in **streaming mode**, the message initiates the loading of multiple elements from memory, generating multiple output AMs. In this mode, the operand address is considered the base address, along with a count to access and load the elements from memory sequentially. These two modes suffice for our benchmarks; however, our architecture allows for integration of additional modes if needed.

**Active Message (AM) Network Interface.** The *AM Network Interface logic* is responsible for injecting AMs into the network. This module comprises two primary components: an *AM Queue* and a *configuration memory*. Each PE contains a dedicated 1KB *AM Queue* (16KB total across the 16-PE array), implemented as a FIFO that stores 70-bit precompiled entries. The *configuration memory*, 10 bits wide, supports up to 8 distinct configurations per PE.

As illustrated in Fig. 8(b), this module performs one of these two operations: (1) If the *output dynamic AM* is available from *Input Network Interface*, the subsequent configuration is loaded from memory based on the *N\_PC* field of the AM (see Fig. 7). This configuration is combined with the *output dynamic AM* and forwarded into the injection port of the router. (2) Alternatively, a *static AM* is injected into the network to keep it occupied. This *static AM* is the concatenation of the next precompiled entry from the *AM Queue* with the first configuration loaded from memory. The generation rate of *static AMs* is determined by the backpressure signal at the router's injection port. The highlighted blue fields in the message format (see Fig. 7) depict data from the configuration memory used to construct the subsequent dynamic AM, with fields *Res\_c*, *Op1\_c*, and *Op2\_c* stored to prevent redundancy.

**3.3.2 Dynamic and Congestion Aware Routing.** *Nexus Machine* supports turn model routing [37], with each router containing five



**Figure 8: Nexus Machine microarchitecture.** A fabric of homogenous PEs interconnected by a mesh network for communicating active messages which carry instructions that can be launched en-route at any PE, enhancing fabric utilization and runtime.

input and five output ports. Specifically, these input ports are designated for messages coming from *AM Network Interface* unit, as well as north, east, south, and west directions, whereas output ports are designated for messages going to *Input Network Interface* unit and four directions. Each input port has a buffer comprising three registers to manage in-flight messages, accompanied by congestion control logic. *Nexus Machine*'s design choice of employing only three registers is motivated by the goal of minimizing overall power consumption. As presented in Fig. 8(c), each router contains a Route Computation Unit, Separable Allocator, and a Crossbar.

**Route Computation** logic considers the destination of messages from all the input ports. It compares it with the positional ID of the PE, and calculates the output port to be requested. This is sent as an input to the allocator. A toy example of **Separable Allocation** process is presented in Fig. 8(d) [37].

**On/Off Congestion control** involves the transmission of a signal to the upstream router when the count of available buffers falls below a threshold, ensuring all in-flight messages will have

buffers on arrival. Each of the five ports transmits an OFF signal when their corresponding available buffer space is reduced to 1, i.e.,  $T_{OFF} = 1$ , and an ON signal when their buffer space reaches 2, i.e.,  $T_{ON} = 2$ . The allocator output controls the 6x5 **Crossbar**.

**3.3.3 Off-chip Memory Datapath.** Each off-chip memory port connects to a row of the PE array via an AXI bus, delivering a combined bandwidth of 1.28GBps. During data loading, data transfers from off-chip memory to the *AM queues* and *data memory* in each PE. The *AM queues* are actively consumed during execution, effectively hiding data loading latency by performing it concurrently with the execution. However, data loading into *data memories* occurs after each tile execution is complete.

**3.3.4 Sparse Metadata Scanners.** The first sparse operand is encoded in *static AMs*. For subsequent sparse operands, a bit-vector scanner hardware assists in efficient iteration over sparse data, providing coordinates within compressed vectors as described in [48]. *Nexus Machine* integrates a modified version of the scanner with

its AXI bus controller to obtain these coordinates. It can handle decoding of vectors of 16 non-zeros and more within 128 elements, allowing it to handle matrices with densities exceeding 12%.

### 3.4 Deadlock avoidance

Deadlock avoidance is a core challenge for message-driven, re-configurable architectures like *Nexus Machine*. Since each PE can generate, process, and reinject AMs at runtime, the NoC must handle dynamic, unpredictable traffic patterns that introduce multiple classes of deadlock. *Nexus Machine* adopts a multi-layered approach that addresses flow-control, routing, and algorithmic deadlocks.

**Flow-control deadlock** arises from circular dependencies among buffers in wormhole-routed networks, where each buffer in a closed loop is full and awaits space in the next. To eliminate this hazard, *Nexus Machine* adopts the Bubble NoC protocol [4], which guarantees at least one empty buffer ("bubble") within any potential cycle. This ensures continuous packet progress and systematically breaks circular waits. However, Bubble NoC alone does not address deadlocks originating from routing path dependencies.

**Routing-level deadlock** occurs when packet routing induces cycles in the channel dependency graph, permitting the possibility of routers mutually waiting, even if buffer space is available. *Nexus Machine* circumvents this scenario by employing the West-First turn model [14], which prohibits specific turns (North to West and South to West). This enforces acyclic channel dependencies and guarantees deadlock-free routing for adaptive message traffic.

**Algorithmic-level deadlock** is unique to *Nexus Machine* due to its dynamic message reinjection mechanism. When an AM is processed by a PE's ALU and subsequently morphed and re-injected into the network, dependency cycles can potentially form. For instance, a message from PE A may travel to PE B for operand retrieval and then return to PE A for result storing; should all relevant output ports be congested, this morphed message may become trapped in a circular wait. To avoid such conditions, *Nexus Machine* statically enforces acyclic data placement and implements a lightweight runtime timeout (1024 cycles) as a safeguard. Across all our evaluated workloads, this timeout mechanism was never invoked.

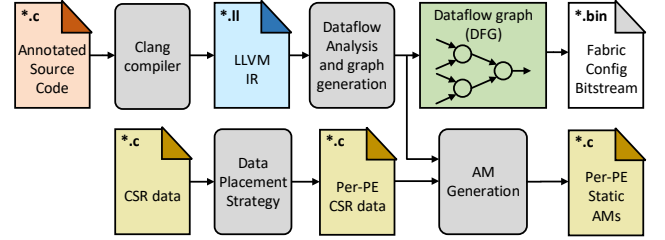
*Nexus Machine* combines Bubble NoC flow control, West-First deadlock-free routing, and acyclic data placement with runtime timeouts to comprehensively avoid deadlock, ensuring forward progress.

### 3.5 Programming Nexus Machine

*Nexus Machine* targets affine loops with associative operations, using the polyhedral model to enable parallel execution, especially for AI workloads involving reduction loops. Programs are written in C, where the programmer annotates independent loop iterations with *parallel-for* keyword, similar to OpenMP or CUDA. The compiler handles nested loops and flattens each annotated iteration, ensuring that instructions within each iteration execute sequentially to preserve intra-iteration dependencies. It is the programmer's responsibility to ensure correct annotations for non-associative inter-iteration dependencies. *Nexus Machine* compiles and executes the program, running each *parallel-for* loop iteration independently

while preserving sequential semantics. Incorrectly annotating dependent loops as *parallel-for* will result in undefined behavior. An example code illustrating these annotations is provided in Fig. 5.

### 3.6 Nexus Machine Compiler



**Figure 9: Process for transforming an application code and data into configuration and data format for *Nexus Machine*.**

*Nexus Machine* integrates a static compiler responsible for pre-processing the application code, alongside a lightweight runtime manager executing on the host processor. The static compiler handles DFG (dataflow graph) generation, data partitioning, and data allocation. Concurrently, the runtime manager processes the data partitions, generating *static AMs* that include the intermediate, final destinations and their designated locations for preloading into the *AM Queues* within the PEs. The dynamic routing is inherently done by *Nexus Machine*'s architecture.

**Static Compiler.** Fig. 9 illustrates the compilation flow for mapping annotated C code and data onto the *Nexus Machine*. The front-end produces LLVM IR, from which a custom pass builds a DFG by identifying instruction dependencies and backedges. The DFG is scheduled using ASAP ordering, and the resulting opcodes are stored in configuration memory. Fig. 5 shows a sample SpMV DFG.

**Problem Definition for Distributed Data Placement:** Effective data placement is critical in this architecture, as it directly impacts performance and resource utilization. A fundamental tradeoff arises: **minimizing communication improves locality but limits possibilities for opportunistic AM execution; increasing communication enables idle PEs to assist via AMs, improving throughput.** Communication is significantly expensive due to network contention and energy constraints, making its reduction a key optimization target.

Given a set of sparse tensors  $A$  and a reconfigurable architecture with  $N$  homogeneous PEs, the objective is to assign tensor elements across PEs to balance memory usage and minimize communication. In distributed memory systems, the non-uniform distribution of nonzeros across rows introduces storage imbalance, which must be carefully managed to ensure scalability and efficiency.

Let each row  $r_i \in R$  have an associated nonzero count  $\text{nnz}(r_i)$ . The goal is to construct a partition  $\{R_1, R_2, \dots, R_N\}$  of rows such that for every processing element  $\text{PE}_k$ , the total number of nonzeros assigned satisfies:  $\sum_{r_i \in R_k} \text{nnz}(r_i) \approx \frac{1}{N} \sum_{r_j \in R} \text{nnz}(r_j)$ . This ensures load balancing across PEs by equalizing the aggregate nonzero count rather than the number of rows but can lead to remote data accesses. *Nexus Machine* exploits the resulting remote accesses using AMs: for remote access, idle PEs on the path may execute the associated AM payload, improving throughput and latency.



**Algorithm 1** Dissimilarity-Aware Data Partitioning

**Require:** CSR tensor  $T$ , number of PEs  $P$ , memory banks  $\mathbb{M}$   
**Ensure:** Mapping  $f_r : R \rightarrow \{PE_1, \dots, PE_P\}$

```

1: for each row  $r_i \in R$  do
2:    $L_i \leftarrow \text{AccessedBanks}(r_i)$  ▷ From column indices
3: end for
4: for each pair  $(r_i, r_j)$  do
5:    $d(i, j) \leftarrow |L_i \Delta L_j|$  ▷ Symmetric difference
6: end for
7:  $\{C_1, \dots, C_P\} \leftarrow \text{Cluster}(R, d(i, j))$  ▷ Group similar rows
8: for each cluster  $C_k$  do
9:   for each  $r \in C_k$  do
10:     $f_r(r) \leftarrow PE_k$  ▷ Assign to PE
11:   end for
12: end for
13: return  $f_r$ 

```

**Dissimilarity-Aware Mapping Strategy:** The primary tensor is partitioned using a dissimilarity-aware strategy [11, 61], shown in Algorithm 1, that spreads memory access and reduces contention, while secondary tensors are co-located or placed nearby to limit inter-PE traffic. Let  $L_i \subseteq \mathbb{M}$  denote memory banks accessed by row  $i$ , and  $d(i, j) = |L_i \Delta L_j|$ . The mapping:  $f_r : R \rightarrow \{PE_1, \dots, PE_P\}$  groups rows with similar  $L_i$  to the same PE and spreads dissimilar ones to avoid contention, enabling efficient AM execution.

**Lightweight Runtime Manager.** The *Runtime Manager*, operating concurrently on the host processor, utilizes data placement information provided by the static compiler to generate a sequence of *static AMs*. These *static AMs* are then loaded into the *AM Queues* of individual PEs for execution. Fig. 7 illustrates the format of these compiler-generated *static AMs*. Each *static AM* corresponds to a unique *Op1* operation and is loaded onto the respective PE, aligned with its designated location. Each *static AM* contains the value of first operand, along with essential details such as the PE ID, and address for the dependent input operands and the result. For every element in the first operand, the runtime manager generates a *static AM* containing information about the operands and the result. The AM pairs this element, stored as *Op1*, with the location of the second operand denoted by *R1*, indicating the PE where it resides, and stores the local address as *Op2*. Similarly, the result is stored at *R2*, along with its local address as *Result*.

## 4 Evaluation Methodology

*Nexus Machine* is implemented in SystemVerilog using the parameters in Table 1, and synthesized with Cadence Genus targeting a commercial 22nm FDSOI process. Memory blocks are instantiated via foundry-validated SRAM compilers for accurate area and timing characterization. The design is fully synthesizable and runs at up to 588 MHz. We develop a cycle-accurate Python simulator that models key behaviors including dynamic routing, congestion, and backpressure. This enables detailed analysis of dataflow and memory access patterns. Application workloads are compiled and mapped onto configuration memory and active message queues.

### 4.1 Baseline Architectures

We evaluate *Nexus Machine* against five baseline architectures. For dense tensor acceleration, we use a **systolic array** modeled after the Google TPU [24] and **systolic array (2:4)** modeled after the Nvidia Tensor Core [8]. The **Generic CGRA** baseline is adapted from HyCube [27], with eight memory banks along two edges to mitigate

| Component   | Configuration                                 |
|-------------|-----------------------------------------------|
| Array       | $4 \times 4$ INT16 array;                     |
| SRAM        | 1KB per PE; 16KB Overall                      |
| AM Queue    | 1KB FIFO with 70 bits per entry, 16KB Overall |
| Main Memory | 4.7GB/s, AXI4 interface                       |

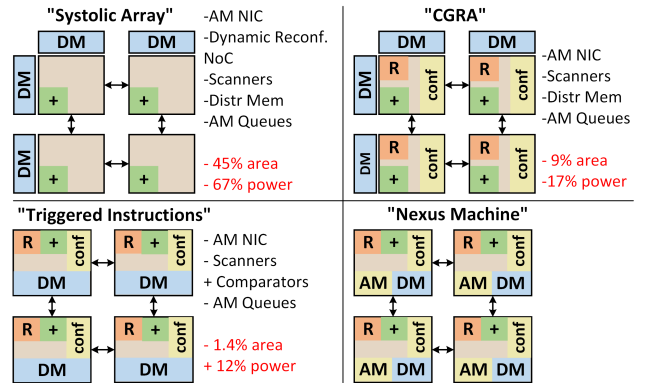
**Table 1: Nexus Machine's architectural parameters.**

memory port limitations. We also evaluate the **TIA** baseline, based on the Triggered Instruction Architecture [45], and a **TIA-Valiant** variant that applies Valiant's randomized minimal-path routing [39, 54] as a traditional network load-balancing scheme. All baselines are synthesized using the same technology node and provisioned with equal ALU counts for matched peak throughput. The *TIA* baseline is evaluated using a cycle-accurate simulator, while CGRA designs use LLVM-based Morpher [56], which models bank conflicts. To ensure fairness, each PE is allocated 2KB on-chip memory for all baselines, while *Nexus Machine* uses 1KB SRAM and 1KB *AM Queue* per PE. We also evaluate the impact of different topologies for *Nexus Machine*, including mesh, torus and ruche configurations. Memories are synthesized using the same SRAM compiler, with layout adapted per architecture as shown in Fig. 10: *Nexus Machine* and *TIA* use distributed memory, while *systolic* and *Generic CGRA* designs employ edge-based shared banks.

### 4.2 Workloads

*Nexus Machine* is designed to accelerate workloads with irregular access patterns, including sparse tensor computations and graph processing. We evaluate *Nexus Machine* across sparse, dense, and graph workloads to demonstrate architectural versatility.

**Sparse Matrix Multiplication (SpMSpM)** multiplies two sparse matrices in Compressed Sparse Row (CSR) format. We implement this using Gustavson's algorithm [61]. To study architectural behavior under varying sparsity, we apply sparsification methods [32, 59] to induce unstructured sparsity while controlling the sparsity-accuracy tradeoff. For stress testing, we evaluate sparsity levels up to 95%, acknowledging that accuracy may degrade beyond 85%. Workloads are categorized as: **S1**: both matrices moderately sparse (30–60%), **S2**: A highly sparse (60–90%), B moderately sparse, **S3**: A moderately sparse, B highly sparse, and **S4**: both highly sparse.



**Figure 10: Ablation of Nexus Machine's features compared to its baselines**

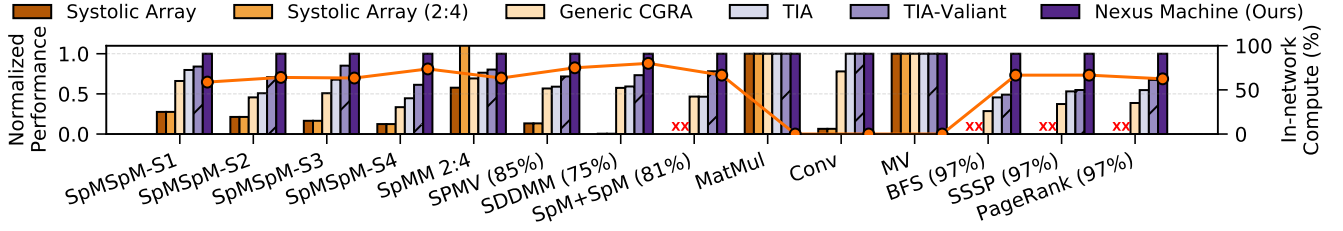


Figure 11: Normalized performance of *Nexus Machine* relative to baselines. The right y-axis shows the percentage of computations performed in-network. Sparsity levels for each workload are noted in parentheses ().

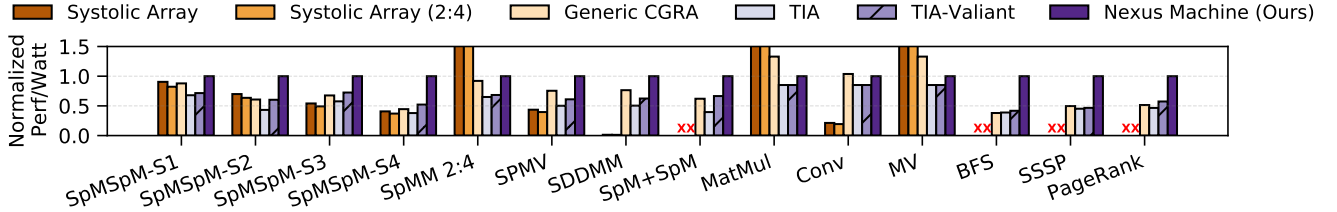


Figure 12: Normalized performance-per-watt of *Nexus Machine* relative to baseline architectures.

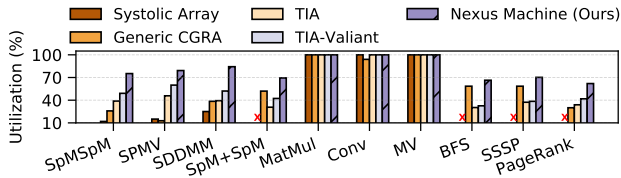


Figure 13: Fabric utilization (in %) of *Nexus Machine* relative to baseline architectures.

Beyond SpMSpM, we evaluate additional sparse workloads: **SpMM-2:4** multiplies a matrix with structured 2:4 sparsity by a dense matrix, aligning with the structured sparsity pattern supported by NVIDIA Tensor Cores [8], **SpMV** multiplies a sparse matrix with a dense vector, **Sparse Matrix Addition (SpM+SpM)** performs element-wise addition of two CSR matrices, common in CNNs, **Sampled Dense-Dense Matrix Multiplication (SDDMM)** computes products only at sparse locations, useful in sparse attention and graph neural networks. For dense workloads, we execute **Matrix Multiplication (Matmul)**, **Matrix Vector Multiplication (MV)**, and **Convolution (Conv)**, all essential for CNNs and language models. For graph analytics, we evaluate **Breadth-First Search (BFS)**, **Single Source Shortest Path (SSSP)**, and **PageRank**, using adjacency list representations.

Benchmarks are mapped to *configuration memory* and *AM queues* in *Nexus Machine*. SpMV, SpMSpM, and SpM+SpM use a pruned and fine-tuned ResNet-50 [20], with convolution layers converted into matrices [6]. SDDMM is evaluated using a sparse mask similar to ViTCoD [59], and dense workloads use unpruned ResNet-50. Graph workloads use infect-dublin [47] dataset with graphs partitioned using Metis [28] for balanced parallel execution.

## 5 Evaluation Results

Fig. 11, Fig. 12 and Fig. 13 show that *Nexus Machine* consistently delivers higher performance, power efficiency and fabric utilization over baseline architectures across workloads with varying sparsity.

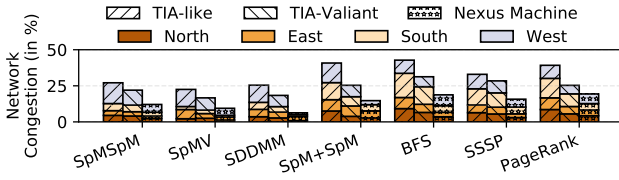
### 5.1 Performance vs. Sparsity

For Sparse workloads, *Nexus Machine* consistently outperforms *systolic*, and *Generic CGRA* across varying sparsity regimes, with performance measured in terms of execution time. For the SpMSpM workload, when the sparsity of the first tensor increases and the others remain fixed, performance decreases due to greater irregularity in AMs within the fabric. In contrast, increasing sparsity in other tensors while keeping the first fixed leads to improved performance, as AMs terminate early when they do not find corresponding elements in the other matrices, reducing computation time. The *Generic CGRA* struggles to execute this kernel efficiently due to a higher number of bank conflicts because of irregular memory accesses and shared bank contention. Intuitively, the bank conflicts in *Generic CGRA* roughly translate to network congestion for *Nexus Machine*, alleviating the entire processing fabric from starving for compute. **The AM driven execution model dynamically adapts to data sparsity and minimizes redundant operations, ultimately enhancing performance under irregular computation patterns.**

For dense workloads, *Nexus Machine*, *TIA* and *TIA-Valiant* baselines emulate systolic-style dataflow for MatMul and MV, achieving performance comparable to the *systolic* and *Generic CGRA* baselines. While the *systolic* delivers the highest performance and best Perf/Watt for these workloads, it is inefficient for Conv due to im2col overhead [7] and cannot execute Conv natively. In contrast, *Nexus Machine* efficiently handles Conv by replicating filters across PEs with minimal overhead. The *systolic (2:4)* baseline outperforms all other designs on the SpMM-2:4 benchmark in both performance

and perf/watt, but underperforms even standard *systolic* baseline on other workloads, as it reverts to dense execution with added hardware overhead for structured sparsity support. *Generic CGRA* achieves near-optimal performance. **Despite being less efficient in Perf/Watt for dense workloads, Nexus Machine's reconfigurable architecture enables robust and versatile execution across diverse dense workloads.**

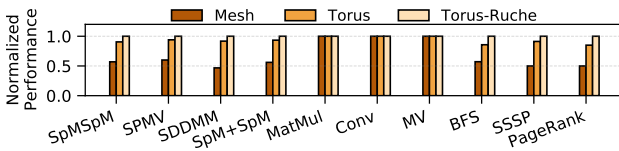
Finally, *TIA* and *TIA-Valiant* baselines, which lack any in-network computation capability, serve as ablation points to distinguish the benefits of en-route computation. The *TIA-Valiant* variant routes to a randomized intermediate destination, distributing traffic evenly across the network, effectively reducing hotspot formations. As shown in Fig. 11, *Nexus Machine* performs a significantly greater portion of its computation on idle PEs en-route, leading to improved fabric utilization. **This showcases the architectural advantage of Nexus Machine in exploiting in-network computation to enhance load balancing and overall system throughput.**



**Figure 14: Network congestion at each input port of the Nexus Machine, relative to TIA baselines. Dense workloads are omitted due to minimal congestion from fixed dataflow patterns.**

Fig. 14 shows network congestion across all input ports, where the *Nexus Machine* reduces average congestion compared to the *TIA* baseline due to its adaptive AM-based routing. Congestion data for *Generic CGRA* is omitted due to its static routing determined at compile time, while *systolic* uses fixed, deterministic routing. This distinction is reflected in compilation time: *Generic CGRA* requires 7.22 seconds for static route resolution, while *Nexus Machine* compiles in 0.55 seconds with runtime routing handled in hardware.

## 5.2 Impact of Network Topology

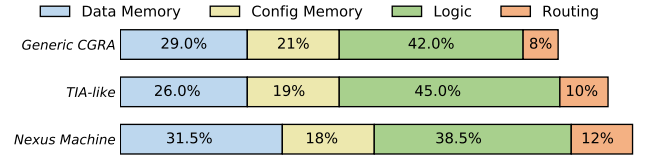


**Figure 15: Normalized performance of Nexus Machine across mesh, torus, and ruche network topologies.**

Fig. 15 presents the normalized performance of *Nexus Machine* with mesh, torus, and torus-ruche topologies across workloads. Torus extends mesh by adding wraparound links for edge PEs along each row and column, increasing bisection bandwidth and reducing average hops, with minimal hardware overhead since only routers of edge PEs have an extra port. Ruche further augments torus by adding two-hop links, providing direct sparse connections between PEs two positions apart.

For *Nexus Machine*, these enhancements are really effective. The message-driven execution model allows AMs to be executed by any idle PE along their route, so advanced topologies help reduce communication distance and improve the likelihood that messages are handled by underutilized PEs. **However, in ruche networks, messages may skip several intermediate PEs, which can occasionally reduce the probability of finding an idle PE for en-route execution; as a result, the overall performance improvement over torus is limited.** The hardware overhead for torus-ruche remains moderate, with area overhead of approximately 5%. **Thus, while richer network topologies increase architectural opportunities for dynamic load balancing and in-network execution, their practical benefit depends on both routing policies and workload characteristics.**

## 5.3 Overhead for AM inspired execution



**Figure 16: Area breakdown of the Nexus Machine compared to Generic CGRA and TIA baselines.**

*Generic CGRA* is used as the baseline to evaluate the impact of architectural enhancements in *Nexus Machine*, specifically the inclusion of a dynamic NoC, AM NIC, and distributed memories instead of the global data memories along the PE array edge for *Generic CGRA*. The *TIA* baseline isolates the contribution of the AM NIC, its associated buffering structures and scanners.

**Area cost:** Fig. 16 compares the area breakdown across *Nexus Machine*, *Generic CGRA*, and *TIA*. *Nexus Machine* exhibits 17.3% and 5.2% area overhead compared to *Generic CGRA* and *TIA*, respectively. Of this, 8% is attributed to the AM queues and related logic. An additional 3% overhead over *Generic CGRA* is due to the integration of scanner units. *TIA* shows an 8% increase relative to *Generic CGRA* due to additional comparators. Both *Nexus Machine* and *TIA* incur a further 6% overhead from dynamic routers implementing turn-model routing and congestion-aware control.

**Power Cost:** Fig. 10 presents the power overhead of *Nexus Machine* relative to the baselines. *Nexus Machine* incurs a 17% increase in total power compared to *Generic CGRA*, with 8% attributed to replicated configuration memories, 0.5% to scanner units, 7% to dynamic routers, and 6% to control logic. In comparison to the *TIA* baseline, *Nexus Machine* incurs 0.5% additional power for scanners but benefits from a 12% reduction in configuration memory power due to the absence of comparators, which contribute significantly to *TIA*'s control logic overhead.

**These results indicate that despite Nexus Machine introducing moderate area and power overheads, its design choices balance flexibility and efficiency by reducing reliance on high-cost control structures.**

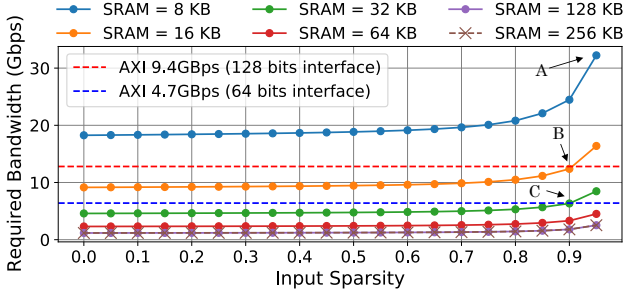


Figure 17: *Nexus Machine* off-chip bandwidth required to achieve peak computational throughput for varying sparsity.

#### 5.4 Data Memory Size vs. Off-Chip Bandwidth

The efficiency of the architecture is affected by the tradeoffs between on-chip memory capacity and off-chip bandwidth, especially across workloads with varying sparsity. To study this relationship, we synthesize and characterize SpMSPM workloads at multiple sparsity levels, which directly affect computation per memory fetch. We apply the sparse tensor partitioning strategy, tiling tensors within on-chip SRAM to minimize off-chip transfers.

As shown in Fig. 17, results reveal a key trade-off: as sparsity increases, computations per byte of off-chip traffic decreases, raising bandwidth requirements, even as *Nexus Machine* sustains stable throughput. Beyond 256KB of on-chip SRAM, bandwidth usage stabilizes at its minimal achievable level. At extreme sparsity ( $\approx 95\%$ ), bandwidth usage rises up to  $7\times$  due to increased output movement, yet dense-equivalent throughput improves up to  $16\times$ , despite partial under-utilization.

*Nexus Machine* was further evaluated using AXI4 interfaces configured at 16 beats with 64-bit and 128-bit widths. These experiments highlight design opportunities based on available AXI4 bandwidth. Design point A, with low on-chip memory requires a higher bandwidth. Conversely, our baseline Design Point B, with substantial on-chip memory, is well suited for workloads with low computation-to-transfer ratios. Lastly, Design Point C can reduce the budget of both on-chip and off-chip memory if the workloads have a higher compute intensity.

#### 5.5 *Nexus Machine*'s Architecture Scalability

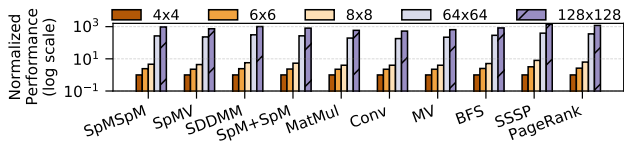


Figure 18: Normalized performance of *Nexus Machine* across increasing array sizes, illustrating its scalability with respect to architectural dimensions.

Fig. 18 evaluates the scalability of *Nexus Machine* across a range of PE array sizes, including large-scale arrays with  $64\times 64$  and  $128\times 128$  PEs, and across various workloads. As the number of PEs increases, *Nexus Machine* demonstrates near linear scaling and maintains high fabric utilization, confirming its ability to efficiently

handle larger problem sizes. For irregular workloads, however, overall efficiency is more sensitive to workload sparsity and data placement than to array size. If operands are placed on opposite sides of a large array, messages must cross the full network, increasing communication latency and congestion, especially in the  $128\times 128$  case. Nonetheless, *Nexus Machine* uses opportunistic en-route execution, so idle PEs along the path can perform computations, helping to offset the cost of long message traversals and supporting sustained throughput at scale. **These results confirm that *Nexus Machine* delivers scalable performance across problem sizes, with utilization and efficiency governed primarily by input sparsity.**

## 6 Related Works

### 6.1 CGRA support for irregularity

Compared to contemporary platforms, CPUs and GPUs are often selected for their flexibility and mature software ecosystems, but both fall short in efficiency when targeting highly irregular workloads relative to domain specific architectures. While GPUs deliver high throughput under regular, data parallel conditions, they experience substantial performance degradation with control divergence and irregular memory access patterns, a challenge that continues to motivate active research [3, 42]. Similarly, High Level Synthesis (HLS) tools for FPGAs are optimized for regular computation and dataflow, limiting their practical effectiveness for irregular workloads despite ongoing research in this area [13, 38]. Unlike FPGAs, which require complete bitstream reconfiguration at the gate level, *Nexus Machine* achieves rapid reconfiguration by updating ALU operations on a cycle-by-cycle basis. In contrast to some prior CGRAs [60] that embed LUTs alongside ALUs within PEs for dynamic instruction sequencing, *Nexus Machine* omits LUTs entirely, simplifying hardware and reducing latency.

*Generic CGRA* architectures are fundamentally inefficient for irregular workloads. Prior research addressing this gap can be broadly divided into server scale approaches, which focus on maximizing memory bandwidth and coarse-grained parallelism, and edge device solutions, which exploit fine-grained parallelism using ultra lightweight hardware mechanisms. *Nexus Machine* is positioned within the latter category, providing architectural support for irregularity through fine-grained, dynamically reconfigurable compute and message-driven execution.

**Server-Scale:** Fifer [40] enables irregular workloads on *generic CGRAs* by decomposing them into regular stages, using queues to decouple and time-multiplex stages for better fabric utilization. However, it is inefficient for edge due to inter/intra-PE queues, frequent reconfiguration, and high data movement costs. Capstan [48] features a checkerboard grid of compute and local memory units, with scanners and address generators for external memory. It uses sparse dataflow patterns and employs sparse-iteration techniques for efficiency and parallelism. Capstan requires additional hardware for data/metadata manipulation and expensive shuffle networks for PEs to access memory. Extensor [21] employs hierarchical tensor intersections in an inner-product flow, exploiting data sparsity but sacrifices programmability, limiting its ability to handle diverse sparse tensor operations efficiently. Dalorex [43] uses a task-based model with uniform data distribution across tiles arranged in a 2D



|                                    | UE-CGRA*<br>[53] | Pipestitch*<br>[50] | Triggered<br>Instructions** [45] | Nexus<br>Machine |
|------------------------------------|------------------|---------------------|----------------------------------|------------------|
| <b>Tech node</b>                   | TSMC 28          | Sub-28              | FDSOI 22                         | FDSOI 22         |
| <b>Setup</b>                       | Post P&R         | Post Synthesis      | Post Synthesis                   | Post Synthesis   |
| <b>Peak Frequency Tested (MHz)</b> | 750              | 50                  | 588                              | 588              |
| <b>Power (mW)</b>                  | 14               | 3.33                | 4.626                            | 3.865            |
| <b>Peak Throughput<br/>(MOPS)</b>  | 625              | 558                 | 490                              | 748              |
| <b>Power Effici.<br/>(MOPS/mW)</b> | 45               | 167                 | 106                              | 194              |

\* Based on the latest values mentioned in Table III of RipTide [16] and approximations from Pipestitch [50] paper.

\*\* Based on our implementation of the work.

**Table 2: Nexus Machine compared to state-of-the-art CGRAs**

torus topology with a ruche network [26]. Each tile includes an in-order core, scratchpad, scheduler, and router. Parent tasks spawn new tasks into the NoC upon completion. *Nexus Machine* balances loads with in-network computation, using idle PEs en-route. While Dalorex works at task-level granularity, ideal for servers, *Nexus Machine* offers instruction-level control for edge devices.

**Edge:** Ultra-Elastic CGRA [53] accelerates inter-iteration loop dependencies in irregular loops by exploiting dynamic voltage and frequency scaling (DVFS) at the granularity of PEs. However, its scope remains constrained by the range of supported workloads. Stardust [22] and TACO [29] provide compiler frameworks for Capstan-like architectures, managing data partitioning and transfers. Sparse Abstract Machine (SAM)[23] offers stream-dataflow abstractions with generic, composable sparse primitives, while SparseTIR[58] explores composable abstractions for sparse deep learning. SAM’s implementation Onyx [30] decouples tensor traversal from dataflows, enabling specialized optimizations as an intermediate representation by combining its primitives. SPU [9] uses a stream dataflow model on a reconfigurable fabric composed of decomposable memories, switches, and compute units that split coarse-grained resources into finer-grained resources on the fabric. This enables flexibility in exploiting common stream-join and indirection data-dependence forms in irregular workloads. Canon [1] employs dynamic orchestration to address single-side sparsity, but its approach does not readily extend to dual-side sparsity or graph workloads. Pipestitch [50] is a CGRA tailored for energy-efficient computation in sensor systems handling sparse ML and DSP tasks. It optimizes performance by converting annotated, parallelizable loops into ordered, pipelined dataflow threads. The design incorporates a dispatch operator that utilizes specialized control flow PEs, forming a synchronization network for ordered-dataflow execution. The architecture demands more PEs compared to *generic CGRAs*, while incurring higher data movement. Table 2 presents a comparison between *Nexus Machine* and state-of-the-art edge CGRAs in terms of power, throughput, and power efficiency. To ensure fairness, we put our best effort into comparing them, considering varied details among different sources. *Nexus Machine* showcases a better power efficiency of 194MOPS/mW compared to the SOTA CGRAs.

## 6.2 Sparse Accelerators

Recent literature introduces Sparse Matrix accelerators tailored for specific workloads. SparTen [18] optimizes bitmap-based inner join operations for SpMM but requires offline load balancing. EIE [19] focuses on CNN fully connected layers but is constrained by on-chip storage. SpAtten [55], for NLP, applies real-time quantization, ranking, and pruning in transformer attention layers. SpArch [62] and

OuterSPACE [44] use outer-product dataflows for SpMSpM but face high partial-sum merging costs. MatRaptor [51] and GAMMA [61] reduce these costs with row-wise product dataflows, and GAMMA adds specialized caching to reduce off-chip transfers. Flexagon [34] dynamically reconfigures dataflows for neural networks based on workload. While these architectures are workload-specific, *Nexus Machine* provides a generic CGRA for efficient execution of both irregular and regular workloads.

## 7 Conclusion

*Nexus Machine* introduces a novel solution for enhancing the acceleration of irregular workloads. By enabling active messages within a reconfigurable architecture and computing on idle PEs in the network, it minimizes data movement and effectively mitigates load-balancing issues. This data-driven execution results in an average of 1.35x performance gain over prior state-of-the-art. Overall, *Nexus Machine* offers a promising architecture for resource-constrained edge devices, striking a balance between high performance, energy efficiency, and adaptability for irregular computational patterns.

## Acknowledgments

We thank the reviewers for their feedback and interesting discussion of this work. This research is supported by the National Research Foundation, Singapore, under its Competitive Research Program Award NRF-CRP23-2019-0003 and the Ministry of Education, Singapore, under Tier 3 grant MOE-MOET32024-0003.

## References

- [1] Zhenyu Bai, Pranav Dangi, Rohan Juneja, Zhaoying Li, Zhanglu Yan, Huiying Lan, and Tulika Mitra. [n. d.]. A Data-Driven Dynamic Execution Orchestration Architecture. <https://www.comp.nus.edu.sg/~tulika/ASPLOS26.pdf> Accessed: 2025-08-06.
- [2] Thilini Kaushalya Bandara, Dan Wu, Rohan Juneja, Dhananjaya Wijerathne, Tulika Mitra, and Li-Shiuan Peh. 2023. FLEX: Introducing FLEXible Execution on CGRA with Spatio-Temporal Vector Dataflow. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. <https://doi.org/10.1109/ICCAD57390.2023.10323612>
- [3] Martin Burtcher, Rupesh Nasre, and Keshav Pingali. 2012. A quantitative study of irregular programs on GPUs. In *2012 IEEE International Symposium on Workload Characterization (IISWC)*. 141–151. <https://doi.org/10.1109/IISWC.2012.6402918>
- [4] Gunnar Carlstedt and Mats Rimborg. 2023. Bubble NoC – A Low Energy Network-on-Chip with Small Footprint and High Performance. (2 2023). <https://doi.org/10.36227/techrxiv.17206295.v2>
- [5] Alex Carsello, Kathleen Feng, Taeyoung Kong, Kalhan Koul, Qiaoyi Liu, Jackson Melchert, Gedeon Nyengele, Maxwell Strange, Keyi Zhang, Ankita Nayak, Jeff Setter, James Thomas, Kavya Sreedhar, Po-Han Chen, Nikhil Bhagdikar, Zachary Myers, Brandon D’Agostino, Pranil Joshi, Stephen Richardson, Rick Bahr, Christopher Torng, Mark Horowitz, and Priyanka Raina. 2022. Amber: A 367 GOPS, 538 GOPS/W 16nm SoC with a Coarse-Grained Reconfigurable Array for Flexible Acceleration of Dense Linear Algebra. In *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 70–71. <https://doi.org/10.1109/VLSITechnologyandCir46769.2022.9830509>
- [6] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014). arXiv:1410.0759 <http://arxiv.org/abs/1410.0759>
- [7] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. arXiv:1410.0759 [cs.NE]. <https://arxiv.org/abs/1410.0759>
- [8] NVIDIA Corporation. 2020. Structured Sparsity in the NVIDIA Ampere Architecture and Applications in Search Engines. <https://developer.nvidia.com/blog/accelerating-inference-with-sparsity-using-ampere-and-tensorrt>. NVIDIA Developer Blog.
- [9] Vidushi Dadu, Jian Weng, Sihao Liu, and Tony Nowatzki. 2019. Towards General Purpose Acceleration by Exploiting Common Data-Dependence Forms. In

- Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 924–939. <https://doi.org/10.1145/3352460.3358276>
- [10] W.J. Dally, J.A.S. Fiske, J.S. Keen, R.A. Lethin, M.D. Noakes, P.R. Nuth, R.E. Davison, and G.A. Fyler. 1992. The message-driven processor: a multicomputer processing node with efficient mechanisms. *IEEE Micro* 12, 2 (1992), 23–39. <https://doi.org/10.1109/40.127581>
- [11] Pranav Dang, Zhenyu Bai, Rohan Juneja, Dhananjaya Wijerathne, and Tulika Mitra. 2024. ZeD: A Generalized Accelerator for Variably Sparse Matrix Computations in ML. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques* (Long Beach, CA, USA) (PACT '24). Association for Computing Machinery, New York, NY, USA, 246–257. <https://doi.org/10.1145/3656019.3689905>
- [12] T.v. Eicken, D.E. Culler, S.C. Goldstein, and K.E. Schauer. 1992. Active Messages: A Mechanism for Integrated Communication and Computation. In *Proceedings of the 19th Annual International Symposium on Computer Architecture*. 256–266. <https://doi.org/10.1109/ISCA.1992.753322>
- [13] Yizhao Gao, Baoheng Zhang, Yuhao Ding, and Hayden So. 2024. A Composable Dynamic Sparse Dataflow Architecture for Efficient Event-based Vision Processing on FPGA. 246–257. <https://doi.org/10.1145/3626202.3637558>
- [14] Christopher J. Glass and Lionel M. Ni. 1992. The turn model for adaptive routing. In *Proceedings of the 19th Annual International Symposium on Computer Architecture* (Queensland, Australia) (ISCA '92). Association for Computing Machinery, New York, NY, USA, 278–287. <https://doi.org/10.1145/139669.140384>
- [15] Graham Gobieski, Ahmet Oguz Atli, Kenneth Mai, Brandon Lucia, and Nathan Beckmann. 2021. Snafu: An Ultra-Low-Power, Energy-Minimal CGRA-Generation Framework and Architecture. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 1027–1040. <https://doi.org/10.1109/ISCA52012.2021.00084>
- [16] Graham Gobieski, Souradip Ghosh, Marijn Heule, Todd Mowry, Tony Nowatzki, Nathan Beckmann, and Brandon Lucia. 2022. RipTide: A Programmable, Energy-Minimal Dataflow Compiler and Architecture. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 546–564. <https://doi.org/10.1109/MICRO56248.2022.00046>
- [17] S.C. Goldstein, H. Schmit, M. Budiu, S. Cadambi, M. Moe, and R.R. Taylor. 2000. PipeRench: a reconfigurable architecture and compiler. *Computer* 33, 4 (2000), 70–77. <https://doi.org/10.1109/2.839324>
- [18] Ashish Gondimalla, Noah Chesnut, Mithuna Thottethodi, and T. N. Vijaykumar. 2019. SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 151–165. <https://doi.org/10.1145/3352460.3358291>
- [19] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 243–254. <https://doi.org/10.1109/ISCA.2016.30>
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [21] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 319–333. <https://doi.org/10.1145/3352460.3358275>
- [22] Olivia Hsu, Alexander Rucker, Tian Zhao, Kunle Olukotun, and Fredrik Kjolstad. 2022. Stardust: Compiling Sparse Tensor Algebra to a Reconfigurable Dataflow Architecture. *CoRR* abs/2211.03251 (2022). <https://doi.org/10.48550/ARXIV.2211.03251>
- [23] Olivia Hsu, Maxwell Strange, Ritvik Sharma, Jaeyeon Won, Kunle Olukotun, Joel S. Emer, Mark A. Horowitz, and Fredrik Kjolstad. 2023. The Sparse Abstract Machine. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 710–726. <https://doi.org/10.1145/3582016.3582051>
- [24] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A. Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. <https://doi.org/10.1145/3579371.3589350>
- [25] Rohan Juneja, Pranav Dang, Thilini Kaushalya Bandara, Zhaoying Li, Dhananjaya Wijerathne, Li-Shiuan Peh, and Tulika Mitra. 2025. Building an Open CGRA Ecosystem for Agile Innovation. [arXiv:2508.19090 \[cs.AR\]](https://arxiv.org/abs/2508.19090) <https://arxiv.org/abs/2508.19090>
- [26] Dai Cheol Jung, Scott Davidson, Chun Zhao, Dustin Richmond, and Michael Bedford Taylor. 2020. Ruche Networks: Wire-Maximal, No-Fuss NoCs : Special Session Paper. In *2020 14th IEEE/ACM International Symposium on Networks-on-Chip (NOCS)*. 1–8. <https://doi.org/10.1109/NOCS50636.2020.9241586>
- [27] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. 2017. HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect. In *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6. <https://doi.org/10.1145/3061639.3062262>
- [28] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multi-level Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing* 20, 1 (1998), 359–392. <https://doi.org/10.1137/S1064827595287997>
- [29] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (oct 2017), 29 pages. <https://doi.org/10.1145/3133901>
- [30] Kalhan Koul, Maxwell Strange, Jackson Melchert, Alex Carsello, Yuchen Mei, Olivia Hsu, Taeyoung Kong, Po-Han Chen, Huifeng Ke, Keyi Zhang, Qiaoyi Liu, Gedeon Nyengele, Akhilesh Balasingam, Jayashree Adivarahan, Ritvik Sharma, Zhouhua Xie, Christopher Tornig, Joel Emer, Fredrik Kjolstad, Mark Horowitz, and Priyanka Raina. 2024. Onyx: A 12nm 756 GOPS/W Coarse-Grained Reconfigurable Array for Accelerating Dense and Sparse Applications. In *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 1–2. <https://doi.org/10.1109/VLSITechnologyandCirc46783.2024.10631383>
- [31] James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratun. 2024. Training-Free Activation Sparsity in Large Language Models. [arXiv:2408.14690 \[cs.CL\]](https://arxiv.org/abs/2408.14690) <https://arxiv.org/abs/2408.14690>
- [32] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A Co-Design Framework for Enabling Sparse Attention Using Reconfigurable Architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 977–991. <https://doi.org/10.1145/3466752.3480125>
- [33] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Hugo De Man, and Rudy Lauwereins. 2003. ADRES: An Architecture with Tightly Coupled VLIW Processor and Coarse-Grained Reconfigurable Matrix. In *International Conference on Field-Programmable Logic and Applications*. <https://api.semanticscholar.org/CorpusID:39182312>
- [34] Francisco Muñoz Martínez, Raveesh Garg, Michael Pellauer, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2023. Flexagon: A Multi-Dataflow Sparse-Sparse Matrix Multiplication Accelerator for Efficient DNN Processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 252–265. <https://doi.org/10.1145/3582016.3582069>
- [35] V. P. Nambiar, Y. S. Chong, T. K. Bandara, D. Wijerathne, Z. Li, R. Juneja, L.-S. Peh, T. Mitra, and A. T. Do. 2024. A 360 GOPS/W CGRA in a RISC-V SoC with Multi-Hop Routers and Idle-State Instructions for Edge Computing Applications. In *2024 21st International SoC Design Conference (ISOC)*. 89–90. <https://doi.org/10.1109/ISOC62682.2024.10762131>
- [36] Vishnu P. Nambiar, Yi Sheng Chong, Thilini Kaushalya Bandara, Dhananjaya Wijerathne, Zhaoying Li, Rohan Juneja, Li-Shiuan Peh, Tulika Mitra, and Anh Tuan Do. 2024. PACE: A Scalable and Energy Efficient CGRA in a RISC-V SoC for Edge Computing Applications. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. 1–1. <https://doi.org/10.1109/HCS61935.2024.10665106>
- [37] Li-Shiuan Peh Natalie Enright Jerger, Tushar Krishna. 2022. *On-Chip Networks, Second Edition*. Springer Cham.
- [38] Reece Neff, Marco Minutoli, Antonino Tumeo, and Michela Becchi. 2023. High-Level Synthesis of Irregular Applications: A Case Study on Influence Maximization. In *Proceedings of the 20th ACM International Conference on Computing Frontiers (Bologna, Italy) (CF '23)*. Association for Computing Machinery, New York, NY, USA, 12–22. <https://doi.org/10.1145/3587135.3592196>
- [39] Ted Nesson and S. Lennart Johnsson. 1995. ROMM routing on mesh and torus networks. In *Proceedings of the Seventh Annual ACM Symposium on Parallel Algorithms and Architectures* (Santa Barbara, California, USA) (SPAA '95). Association for Computing Machinery, New York, NY, USA, 275–287. <https://doi.org/10.1145/215399.215455>
- [40] Quan M. Nguyen and Daniel Sanchez. 2021. Fifer: Practical Acceleration of Irregular Applications on Reconfigurable Architectures. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 1064–1077. <https://doi.org/10.1145/3466752.3480048>
- [41] Tony Nowatzki, Vinay Gangadhar, Newsha Ardalani, and Karthikeyan Sankaralingam. 2017. Stream-dataflow acceleration. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. 416–429. <https://doi.org/10.1145/3079856.3080255>
- [42] Molly A. O'Neil and Martin Burtcher. 2014. Microarchitectural performance characterization of irregular GPU kernels. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*. 130–139. <https://doi.org/10.1109/IISWC>

- 2014.6983052
- [43] Marcelo Orenes-Vera, Esin Tureci, David Wentzlaff, and Margaret Martonosi. 2022. Dalorex: A Data-Local Program Execution and Architecture for Memory-bound Applications. *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2022), 718–730. <https://api.semanticscholar.org/CorpusID:251105150>
  - [44] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Apurva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 724–736. <https://doi.org/10.1109/HPCA.2018.00067>
  - [45] Angshuman Parashar, Michael Pellauer, Michael Adler, Bushra Ahsan, Neal Crago, Daniel Lustig, Vladimir Pavlov, Antonia Zhai, Mohit Gambhir, Aamer Jaleel, Randy Allmon, Rachid Rayess, Stephen Maresh, and Joel Emer. 2014. Efficient Spatial Processing Element Control via Triggered Instructions. *IEEE Micro* 34, 3 (2014), 120–137. <https://doi.org/10.1109/MM.2014.14>
  - [46] Raghu Prabhakar, Yaqi Zhang, David Koeplinger, Matt Feldman, Tian Zhao, Stefan Hadjis, Ardavan Pedram, Christos Kozyrakis, and Kunle Olukotun. 2017. Plasticine: A reconfigurable architecture for parallel patterns. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. 389–402. <https://doi.org/10.1145/3079856.3080256>
  - [47] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI*. <https://networkrepository.com>
  - [48] Alexander Rucker, Matthew Vilim, Tian Zhao, Yaqi Zhang, Raghu Prabhakar, and Kunle Olukotun. 2021. Capstan: A Vector RDA for Sparsity. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) (*MICRO '21*). Association for Computing Machinery, New York, NY, USA, 1022–1035. <https://doi.org/10.1145/3466752.3480047>
  - [49] K. Sankaralingam, R. Nagarajan, Haiming Liu, Changkyu Kim, Jaehyuk Huh, D. Burger, S.W. Keckler, and C.R. Moore. 2003. Exploiting ILP, TLP, and DLP with the polymorphous TRIPS architecture. In *30th Annual International Symposium on Computer Architecture, 2003. Proceedings.* 422–433. <https://doi.org/10.1109/ISCA.2003.1207019>
  - [50] Nathan Serafin, Souradip Ghosh, Harsh Desai, Nathan Beckmann, and Brandon Lucia. 2023. Pipestitch: An energy-minimal dataflow architecture with light-weight threads. *IEEE Micro* (2023).
  - [51] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 766–780. <https://doi.org/10.1109/MICRO50266.2020.00068>
  - [52] S. Swanson, K. Michelson, A. Schwerin, and M. Oskin. 2003. WaveScalar. In *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36.* 291–302. <https://doi.org/10.1109/MICRO.2003.1253203>
  - [53] Christopher Torng, Peitian Pan, Yanghui Ou, Cheng Tan, and Christopher Batten. 2021. Ultra-Elastic CGRAs for Irregular Loop Specialization. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 412–425. <https://doi.org/10.1109/HPCA51647.2021.00042>
  - [54] L. G. Valiant and G. J. Brebner. 1981. Universal schemes for parallel communication. In *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing* (Milwaukee, Wisconsin, USA) (*STOC '81*). Association for Computing Machinery, New York, NY, USA, 263–277. <https://doi.org/10.1145/800076.802479>
  - [55] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 97–110.
  - [56] Dhananjaya Wijerathne, Zhaoying Li, Manupa Karunaratne, Li-Shiuan Peh, and Tulika Mitra. 2022. Morpher: An Open-Source Integrated Compilation and Simulation Framework for CGRA. In *Workshop on Open-Source EDA Technology (WOSET)*.
  - [57] Zi Yu Xue, Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. 2023. Tailors: Accelerating Sparse Tensor Algebra by Overbooking Buffer Capacity. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) (*MICRO '23*). Association for Computing Machinery, New York, NY, USA, 1347–1363. <https://doi.org/10.1145/3613424.3623793>
  - [58] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. Sparse-TIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (<conf-loc>, <city>Vancouver</city>, <state>BC</state>, <country>Canada</country>, </conf-loc>)* (*ASPLOS 2023*). Association for Computing Machinery, New York, NY, USA, 660–678. <https://doi.org/10.1145/3582016.3582047>
  - [59] Haoran You, Zhanyi Sun, Huihong Shi, Zhongzhi Yu, Yang Zhao, Yongan Zhang, Chaofan Li, Baopu Li, and Yingyan Lin. 2023. ViTCoD: Vision Transformer Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 273–286. <https://doi.org/10.1109/HPCA56546.2023.10071027>
  - [60] Baofen Yuan, Jianfeng Zhu, Xingchen Man, Zijiao Ma, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2022. Dynamic-II Pipeline: Compiling Loops With Irregular Branches on Static-Scheduling CGRA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 9 (2022), 2929–2942. <https://doi.org/10.1109/TCAD.2021.3121346>
  - [61] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (*ASPLOS '21*). Association for Computing Machinery, New York, NY, USA, 687–701. <https://doi.org/10.1145/3445814.3446702>
  - [62] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 261–274. <https://doi.org/10.1109/HPCA47549.2020.00030>
  - [63] Ningxin Zheng, Huiqiang Jiang, Quanlu Zhang, Zhenhua Han, Yuqing Yang, Lingxiao Ma, Fan Yang, Chengruidong Zhang, Lili Qiu, Mao Yang, and Lidong Zhou. 2023. PIT: Optimization of Dynamic Sparse Deep Learning Models via Permutation Invariant Transformation. *arXiv:2301.10936 [cs.LG]* <https://arxiv.org/abs/2301.10936>